



Course Details

Course Title: Data Mining

Couse Code: CSE 477

Section: 02

Submitted To

[Amit Mandal](#)

Senior Lecturer

Department of Computer Science and Engineering

Submitted By

Farhatun Nahar Priya

Student ID: 2023-1-60-202

Date of Submission: 26th January 2026.

Table of Contents

1. Objective	2
2. Tools and Libraries Used	3
2.1 Programming Language	3
2.2 Installed and Imported Libraries	3
2.3 Additional Python Modules	3
2.4 NLTK Datasets Downloaded	4
2.5 Execution Environment	4
3. Dataset Description	4
3.1 comments.txt	4
4.2 captions.vtt	5
5. Methodology / Procedure	6
5.1 Profiling the Data	6
5.2 Parsing Comments and Captions	6
5.2.1 Parsing Comments	6
5.2.2 Parsing Captions	6
5.3 Cleaning and Preprocessing Pipeline	7
5.3.1 Normalization	7
5.3.2 Tokenization	7
5.3.3 Stopword Removal	7
5.3.4 Lemmatization	7
5.4 Exporting Cleaned Data	7
6. Code Implementation	8
6.1 Parsing Comments	8
6.2 Parsing Captions	9
6.3 Cleaning Functions	10
6.4 Combined Cleaning Pipeline	11
6.5 Exporting Cleaned Data	12
7. Results and Discussion	13
7.1 Results	13

7.2 Observations	13
7.3 Challenges Faced	13
7.4 Improvement in Data Quality	14
8. Conclusion.....	14
9. References	14

1. Objective

The primary objective of this lab is to clean and preprocess raw YouTube textual data to make it suitable for text mining and natural language processing applications. This lab involves transforming unstructured YouTube comments and caption files into structured formats using Python-based techniques. Standard preprocessing steps such as normalization, tokenization, stopwords removal and lemmatization are applied to reduce noise and improve the overall quality of the text data. The cleaned and structured datasets prepared in this lab will be used in future experiments, including sentiment analysis, topic modelling and frequency-based text analysis.

2.Tools and Libraries Used

2.1 Programming Language

- **Python 3.10** – Python 3.10 was used as the primary programming language for this lab due to its simplicity, readability and strong support for text processing and data analysis tasks. All codes were executed inside the Kaggle Notebook environment.

2.2 Installed and Imported Libraries

- **pandas** – Used for creating and managing DataFrames, reading and writing CSV files and performing various tabular data operations. It was mainly used to organize parsed comments and captions into structured formats.
- **nltk** – Used for natural language processing tasks such as tokenization, stopwords removal, stemming and lemmatization to improve text quality and consistency.
- **webvtt-py** – Used for reading and parsing .vtt caption files extracted from YouTube videos, allowing the extraction of meaningful caption text while ignoring timestamps and metadata.
- **matplotlib** – Used for simple and optional visualizations during data inspection and analysis.

2.3 Additional Python Modules

- **re** – Used for cleaning and structuring raw text using regular expressions, such as detecting timestamps, removing special characters, and handling inconsistent formatting.
- **os** – Used for file and directory handling operations, including creating folders and managing file paths for saving cleaned datasets.

- **datetime (timedelta)** – Used for handling and manipulating timestamp-related information when necessary.
- **collections.Counter** – Used for counting words, analyzing token frequencies, and generating basic summary statistics.
- **nltk.bigrams** – Used for experimenting with basic bigram (word pair) generation and exploring simple language patterns.

2.4 NLTK Datasets Downloaded

- **punkt** – Used for sentence and word tokenization. It helps in accurately splitting text into individual words and sentences.
- **stopwords** – Contains a list of commonly used English words (such as *is*, *the*, and *and*) that are removed during preprocessing to reduce noise.
- **wordnet** – A lexical database used for lemmatization, allowing words to be converted into their base or dictionary form.
- **omw-1.4** – An extension of WordNet that improves lemmatization support and word sense mapping across different forms.
- **punkt_tab** – Used internally by the tokenizer to improve tokenization accuracy and language handling.

2.5 Execution Environment

- The entire data cleaning and preprocessing pipeline was implemented and executed within the Kaggle Notebook environment.
- To enable caption file processing, the following command was executed in Kaggle:

```
!pip install webvtt-py
```

3. Dataset Description

This lab uses two different datasets collected from YouTube to perform data cleaning and preprocessing tasks. Both datasets contain raw and unstructured textual data that require processing before being used for text mining analysis.

3.1 comments.txt

The comments.txt file contains raw YouTube comment data collected from a video. Each comment entry generally includes a username, a relative timestamp (such as “2 days ago” or

“3 hours ago”), and the comment text posted by the user. The comments are typically stored as repeated blocks of text rather than in a structured format.

@milojadez

14 hours ago

There are people out there that actually are able to read minds and solve murders unexplainably, this guy just found his way to gain from it.

Reply

@JasonWestaway

22 hours ago

Wow, that was insane!

Reply

Irregularities noticed during inspection:

- Blank lines between comments
- UI leftovers like "Reply", "...more", "View X replies"
- Mixed languages in some comments (English + Bangla)
- Comments sometimes span multiple lines

4.2 captions.vtt

The captions.vtt file contains subtitle or caption data extracted from a YouTube video in WebVTT format. The file includes timestamp information along with caption text that represents the spoken content of the video.

WEBVTT

00:00:04.240 --> 00:00:06.470

I am build as the world's greatest mind

00:00:06.480 --> 00:00:09.190

readader. But guess what? I can't read minds.

Irregularities noticed during inspection:

- WEBVTT header and timestamps present
- Blank lines between subtitle chunks
- Sentence boundaries do not always align with timing chunks
- Text sometimes contains formatting tags like <c>

5. Methodology / Procedure

5.1 Profiling the Data

Initially, both comments.txt and captions.vtt files were opened and inspected manually to understand their structure and content.

- For comments.txt, the structure of each comment was analyzed to identify usernames, timestamps, and comment text.
 - Observed issues included blank lines, lines containing “Reply” or “...more”, multi-line comments, and inconsistent formatting.
 - For captions.vtt, timestamp lines, metadata, and formatting tags such as <c> and <i> were identified. These elements were marked for removal to extract only meaningful caption text.
- This profiling step provided a clear understanding of the raw data and guided the parsing and cleaning procedures.

5.2 Parsing Comments and Captions

5.2.1 Parsing Comments

- A regular expression was applied to detect timestamp lines (e.g., “2 minutes ago”, “1 day ago”).
- Each comment was grouped into a structured format containing:
 - username
 - timestamp_text
 - comment_text
- Non-informative lines such as “Reply” or “...more” were ignored.

- The resulting structured data was stored in a pandas DataFrame, making it suitable for subsequent cleaning and preprocessing steps.

5.2.2 Parsing Captions

- Timestamp lines and metadata in .vtt files were skipped.
- Only meaningful caption text was extracted.
- Caption lines were merged and further split into individual sentences.
- A DataFrame was created with a single column:
 - caption_sentence
- This structured format allowed efficient cleaning and tokenization.

5.3 Cleaning and Preprocessing Pipeline

A consistent preprocessing pipeline was applied to both comments and captions, consisting of the following steps:

5.3.1 Normalization

- All text was converted to lowercase.
- Punctuation, numbers, and special characters were removed.
- VTT formatting tags such as <c> and <i> were eliminated.
- Bracketed expressions like [laugh] or [applause] were also removed.

5.3.2 Tokenization

- Text was split into individual words using NLTK's word_tokenize(), allowing further analysis at the word level.

5.3.3 Stopword Removal

- Common English stopwords (e.g., *is*, *the*, *and*) were removed to reduce noise.
- Very short tokens (length ≤ 2) were also ignored as they typically do not carry meaningful information.

5.3.4 Lemmatization

- Words were converted into their base form using WordNetLemmatizer.
- Lemmatization was preferred over stemming to preserve the semantic meaning of words.

This pipeline ensured that both comments and captions were converted into a clean, consistent, and analysis-ready format.

5.4 Exporting Cleaned Data

- The cleaned comments DataFrame was saved as `cleaned_comments.csv`.
- The cleaned captions DataFrame was saved as `cleaned_captions.csv`.
- Both files were stored inside a dedicated folder named `cleaned_data` for organized access in future labs.

This step completed the preprocessing workflow, resulting in structured and clean textual datasets suitable for text mining and NLP tasks.

6. Code Implementation

The code was divided into the following sections for clarity:

6.1 Parsing Comments

Purpose:

Convert raw YouTube comments from a `.txt` file into a structured DataFrame.

How it works:

- Reads all lines from the file
- Detects timestamps using a regular expression
- Treats the line before the timestamp as the username
- Collects the comment text, ignoring blank lines and UI leftovers
- Joins multi-line comments into a single string
- Stores all comments in a pandas DataFrame

Code:

```
def structure_comments_from_txt(filepath): with open(filepath, 'r', encoding='utf-8') as f:
lines = f.readlines()
comments_data = []
time_regex = re.compile(r'.*(second|minute|hour|day|week|month|year)s?\s+ago.*',
re.IGNORECASE)
```

```

i = 0
while i < len(lines):
    line = lines[i].strip()

    if i+1 < len(lines) and time_regex.match(lines[i+1].strip()):
        username = line
        timestamp = lines[i+1].strip()
        comment_text = []

        i += 2
        while i < len(lines) and not (
            i+1 < len(lines) and time_regex.match(lines[i+1].strip())
        ):
            text = lines[i].strip()
            if text and text.lower() not in ['reply', '...more']:
                comment_text.append(text)
            i += 1

        if comment_text:
            comments_data.append({
                'username': username,
                'timestamp_text': timestamp,
                'comment_text': ''.join(comment_text)
            })

        else:
            i += 1

return pd.DataFrame(comments_data)
comments_df = structure_comments_from_txt('/kaggle/input/comments-and-
caption/Comments.txt')
comments_df.head()

```

6.2 Parsing Captions

Purpose:

Convert YouTube .vtt subtitle files into structured sentences for analysis.

How it works:

- Reads .vtt files line by line
- Ignores timestamps, numeric indices, and metadata (WEBVTT)
- Joins the caption lines into a single text
- Splits the text into sentences
- Returns a pandas DataFrame with each sentence as a row

Code:

```
def structure_captions_from_vtt(filepath):
    captions = []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            # Skip timestamps, blank lines, numeric indices, and header
            if '-->' not in line and line and not line.isdigit() and 'WEBVTT' not in line:
                captions.append(line)

    full_text = ''.join(captions)
    # Split text into sentences
    sentences = re.split(r'(?<=[!?!]) +', full_text)
    return pd.DataFrame(sentences, columns=['caption_sentence'])

captions_df = structure_captions_from_vtt('/kaggle/input/comments-and-caption/Caption.vtt')
captions_df.head()
```

6.3 Cleaning Functions

Purpose:

Clean and preprocess text data for analysis by performing:

- Normalization (lowercasing, removing brackets and VTT tags, keeping only letters)
- Tokenization (splitting text into words)
- Stopword removal (removing common irrelevant words)
- Lemmatization / Stemming (reducing words to their base or root form)

Code:

```

# Initialize stopwords, lemmatizer, and stemmer
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
stemmer = PorterStemmer()

# Remove common VTT tags
def remove_vtt_tags(text):
    return re.sub(r'<.*?>', '', text)

# Normalize text
def normalize_text(text):
    text = text.lower()
    text = re.sub(r'\[.*?\]', '', text)    # remove brackets like [laugh]
    text = remove_vtt_tags(text)          # remove VTT tags
    text = re.sub(r'^a-z\s', '', text)    # keep only letters and spaces
    return text.strip()

# Tokenize text
def tokenize_text(text):
    return word_tokenize(text)

# Remove stopwords
def remove_stopwords(tokens):
    return [word for word in tokens if word not in stop_words and len(word) > 2]

# Lemmatize tokens
def lemmatize_tokens(tokens):
    return [lemmatizer.lemmatize(word) for word in tokens]

# Stem tokens
def stem_tokens(tokens):
    return [stemmer.stem(word) for word in tokens]

```

6.4 Combined Cleaning Pipeline

Purpose:

Combine all text preprocessing steps into a single function for consistent cleaning of both comments and captions.

How it works:

- Normalizes the text (lowercase, remove VTT tags and brackets)
- Tokenizes into words
- Removes stopwords
- Applies lemmatization or stemming
- Can be applied to any text column (comments or captions)

Code:

```
# Combined cleaning pipeline
def clean_text_pipeline(text, use_lemmatization=True):
    normalized = normalize_text(text)
    tokens = tokenize_text(normalized)
    filtered = remove_stopwords(tokens)

    if use_lemmatization:
        return lemmatize_tokens(filtered)
    else:
        return stem_tokens(filtered)

# Apply the pipeline to comments
comments_df['cleaned_tokens'] = comments_df['comment_text'].apply(
    lambda x: clean_text_pipeline(x, use_lemmatization=True)
)

# Preview the cleaned tokens
comments_df.head()
```

6.5 Exporting Cleaned Data

Purpose:

Save the cleaned and preprocessed comments and captions to CSV files for further analysis.

How it works:

- Uses pandas `to_csv()` function to export DataFrames
- Ensures the data can be easily reused or shared
- Keeps indices optional for a cleaner file

```

import os
folder_path = '/kaggle/working/cleaned_data'
os.makedirs(folder_path, exist_ok=True)
print("Folder exists:", os.path.exists(folder_path))
comments_df.to_csv(f'{folder_path}/cleaned_comments.csv', index=False)
captions_df.to_csv(f'{folder_path}/cleaned_captions.csv', index=False)
print("Files in cleaned_data folder:", os.listdir(folder_path))

```

7. Results and Discussion

7.1 Results

- Successfully parsed valid comments and caption sentences
- Generated cleaned token lists for each entry
- Exported structured and cleaned CSV files

	username	timestamp_text	comment_text	cleaned_tokens
0	@milojadez	14 hours ago	There are people out there that actually are a...	[people, actually, able, read, mind, solve, mu...
1	@JasonWestaway	22 hours ago	Wow, that was insane! 1	[wow, insane]
2	@THEHONESTREPLAY	1 day ago	That's all are fake, Edited, i challenged,	[thats, fake, edited, challenged]
3	@serenaLei06	1 day ago	Wait, how did he do that?	[wait]
4	@ainomugishanancy206	1 day ago	If i know how people think,i know what they th...	[know, people, thinki, know, think]

Figure : Sample of YouTube comments with cleaned tokens.

	caption_sentence	cleaned_tokens
0	Kind: captions Language: en l<00:00:04.560><c>...	[kind, caption, language, build, world, greate...
1	But guess what?	[guess]
2	I can't read readader.	[cant, read, readader]
3	But guess what?	[guess]
4	I can't read minds.<00:00:10.480><c> What</c><...	[cant, read, mind, read, people, mind]

Figure: Sample of YouTube caption with cleaned tokens.

7.2 Observations

- Many unnecessary words and symbols were removed
- Cleaned text is more readable and suitable for analysis
- Stopword removal significantly reduced noise
- Lemmatization helped in unifying word forms

7.3 Challenges Faced

- Inconsistent comment formatting
- Multi-line comments
- Special characters and mixed languages
- Caption tags and metadata

7.4 Improvement in Data Quality

- Raw data was noisy and unstructured
- Cleaned data is structured, consistent, and analysis-ready
- Preprocessing improved accuracy for future NLP tasks

8. Conclusion

In this lab, raw YouTube comments and caption data were successfully cleaned and preprocessed using a systematic text preprocessing pipeline. Unstructured and noisy textual data were transformed into clean, structured and organized formats suitable for analysis. Various preprocessing techniques such as normalization, tokenization, stopwords removal and lemmatization helped reduce noise, remove irrelevant information and improve overall text consistency. This conversion from raw text to cleaned data is a crucial step for text mining and natural language processing tasks, as it enhances data quality and readability.

9. References

1. NLTK Documentation

Available at: [NLTK :: Natural Language Toolkit](https://www.nltk.org/)

This documentation was used to understand and implement text preprocessing techniques such as tokenization, stopwords removal, and lemmatization.

2. Pandas Documentation

Available at: <https://pandas.pydata.org/docs/>

This resource was consulted for creating and managing DataFrames, handling tabular data, and exporting cleaned datasets to CSV files.

3. Python re Module Documentation

Available at: <https://docs.python.org/3/library/re.html> This documentation was used for applying regular expressions to clean raw text, detect patterns such as timestamps, and handle inconsistent formatting.

4. webvtt-py Documentation

Available at: <https://github.com/glut23/webvtt-py>

This reference was used for reading and parsing .vtt caption files and extracting meaningful caption text while ignoring timestamps and metadata.